PAI-1 IntegriDos

Verificadores de Integridad en la Transmisión para Entidad Financiera (SecBank)

Ingeniería de Seguridad · Universidad de Sevilla
Grupo 20 — Security Team

Integrante	Rol en el proyecto
Ricardo Guisado Arenas	Rol A — Gestión de usuarios y credenciales (RS1)
Álvaro Morillo Navajas	Rol B — Integridad y anti-replay (RS2 / RS3)
Francisco Javier Rodríguez Martín	Rol C — Canal lateral y evidencias (RS4 / capturas)

1. Resumen ejecutivo

SecBank es un sistema cliente-servidor para transferencias bancarias que garantiza integridad, autenticidad y protección frente a reintervención sin depender de la capa de transporte (TLS/HTTPS está explícitamente prohibido por el enunciado). Toda la seguridad se implementa en la capa de aplicación: las credenciales se almacenan con Argon2id y salt aleatorio, cada transacción se firma con HMAC-SHA256 sobre una representación JSON canónica usando una clave de sesión de 256 bits, se protege frente a reenvío mediante nonces de un solo uso y ventana temporal, y la verificación de la firma se realiza con comparación en tiempo constante para evitar canales laterales de temporización.

El sistema se ha implementado en Python con FastAPI (servidor HTTP en texto plano sobre el puerto 8080) y SQLite/SQLAlchemy como persistencia. Se han generado evidencias reales de red (capturas .pcapng con Wireshark) para los ataques de integridad (RS2) y de replay (RS3), y una auditoría de código más un experimento empírico de tiempos para la resistencia a canal lateral (RS4). Todos los tests automatizados (RS2/RS3) pasan correctamente.

2. Arquitectura y stack tecnológico

Componente	Elección	Motivo
Transporte	API REST sobre HTTP plano (FastAPI, puerto 8080)	Opción B del enunciado; TLS prohibido explícitamente
Servidor	Python 3.13 + FastAPI + Uvicorn	Tipado con Pydantic, documentación automática (/docs)
Persistencia	SQLite + SQLAlchemy (ORM)	Sin dependencias externas, suficiente para el alcance del PAI
Credenciales	Argon2id (argon2-cffi)	Función de derivación adaptativa recomendada por el enunciado
Integridad	HMAC-SHA256, clave de sesion de 256 bits	RS2 - PRNG criptografico del sistema operativo
Anti-replay	Nonce de 128 bits + ventana temporal de 30s	RS3 — tabla ProcessedNonce con purga periódica
Comparación de firmas	secrets.compare_digest()	RS4 — tiempo constante

3. Diseño del protocolo — diagramas de secuencia

3.1 Login

```
Cliente Servidor
| POST /login {username, password} |
|--------------------------------------------------- >|
| Verifica Argon2id, rate limiting (RS1) |
| 200 OK {session_token, session_key(256b), iban} |
|<--------------------------------------------------- |
```

3.2 Transferencia legítima

```
Cliente Servidor
| firma = HMAC-SHA256(session_key, nonce+ts+JSON) |
| POST /transfer [X-Session-Token, X-Signature, |
| X-Nonce, X-Timestamp] + body JSON |
|--------------------------------------------------- >|
| 1) sesion valida 2) timestamp en ventana (+-30s) |
| 3) nonce no repetido (RS3) 4) firma valida (RS2) |
| comparacion en tiempo constante (RS4) |
| 200 OK {transfer_id, status: success} |
|<--------------------------------------------------- |
```

3.3 Ataque de integridad (mensaje alterado)

```
Atacante Servidor
| POST /transfer amount alterado (25.50 -> 500.00) |
| firma calculada SIN conocer la session_key real |
|--------------------------------------------------- >|
```

```
| firma_esperada != firma_recibida |
| 401 Unauthorized - Fallo de integridad |
|<------------------------------------------------- |
```

3.4 Ataque de replay

```
Atacante Servidor
| Reenvia la peticion legitima 3.2, byte a byte igual |
| (mismo nonce, misma firma, mismo body) |
|-------------------------------------------------- >|
| nonce ya existe en ProcessedNonce |
| 400 Bad Request - Ataque Replay detectado |
|<------------------------------------------------- |
```

4. Análisis de seguridad y evidencias

4.1 RS1 — Almacenamiento y verificación de credenciales

Las contraseñas se hashean con Argon2id (`hash_password()`, salt aleatorio automático gestionado por la propia librería). El login usa un hash *dummy* precalculado cuando el usuario no existe, para que el tiempo de respuesta no revele si una cuenta está o no registrada. Tras 5 intentos fallidos consecutivos la cuenta se bloquea 60 segundos (`423 Locked`). El registro (`POST /register`) impide duplicados tanto de *username* como de IBAN, devolviendo `409 Conflict` en ambos casos.

4.2 RS2 — Integridad y autenticidad de transacciones

Cada transferencia se firma con HMAC-SHA256 sobre el mensaje canónico `nonce + timestamp + JSON(sort_keys=True, sin espacios)`, usando la `session_key` de 256 bits generada con `secrets.token_hex(32)` (PRNG criptográfico). Evidencia de red capturada con Wireshark (interfaz de loopback, filtro `tcp.port == 8080`):

```
Petición legítima — X-Signature: 9337820d7368...af6c4e1ac · X-Nonce: 7862b33f5b5e... · body: {origin_iban, destination_iban, amount: 25.5, concept} -> 200 OK.
Petición alterada — mismo flujo pero amount: 500.0 y firma recalculada sin conocer la session_key real -> la firma no coincide -> 401 Unauthorized: "Fallo de integridad: La firma HMAC no coincide o el mensaje fue alterado."
```

Fichero de evidencia: `captures/02_evidencias_rs2_rs3.pcapng` *(frame 23: petición legítima firmada · frame 33: 200 OK · frame 42: misma transacción con* `amount` *alterado a 500.00 · frame 50: 401 Unauthorized).*

4.3 RS3 — Protección contra ataques de replay

Cada mensaje incluye un nonce aleatorio de 128 bits en la cabecera `X-Nonce`. El servidor mantiene la tabla `ProcessedNonce` con los nonces ya usados dentro de la ventana de 30 segundos, purgando los expirados en cada petición. Prueba: se reenvía la petición legítima del apartado 4.2 byte a byte idéntica (mismo nonce, misma firma, mismo body).

```
Petición reenviada — idéntica a la legítima original -> 400 Bad Request: "Ataque Replay detectado: Este nonce ya ha sido procesado."
```

Fichero de evidencia: `captures/02_evidencias_rs2_rs3.pcapng` (frame 57: reenvío byte a byte de la petición del frame 23, mismo `X-Signature` y `X-Nonce` · frame 63: 400 Bad Request). A diferencia del caso 4.2, aquí la firma SÍ es válida — el rechazo viene exclusivamente de la detección de nonce repetido, demostrando que RS3 actúa de forma independiente a RS2.

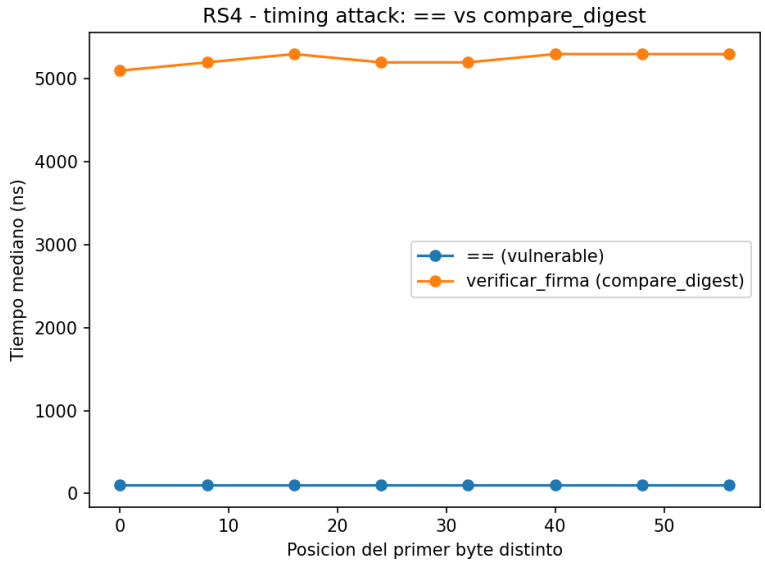
4.4 RS4 — Mitigación de canales laterales de tiempo

La verificación de firma (`verificar_firma()` en `app/security.py`) usa `secrets.compare_digest()` en lugar del operador `==`. Auditoría estática del código (búsqueda de todas las comparaciones `==`): las únicas coincidencias son filtros de consulta SQL (SQLAlchemy), no comparaciones de secretos en memoria.

Experimento empírico: se mide, con la mediana de 3000 repeticiones, el tiempo de comparar una firma correcta frente a firmas incorrectas que fallan en distintas posiciones, usando (a) `==` y (b) `verificar_firma()` real.

Posición byte incorrecto	== a pelo (ns)	verificar_firma() (ns)
0	100	5100

8	100	5200
16	100	5300
24	100	5200
32	100	5200
40	100	5300
48	100	5300
56	100	5300



Comparación == (vulnerable, tiempo variable) vs verificar_firma() (tiempo constante).

Conclusión: contra lo esperado teóricamente, el tiempo medido de == se mantiene plano (~100 ns) en todas las posiciones probadas, en lugar de crecer con la distancia del primer byte distinto. Esto no invalida RS4: a este tamaño de cadena (64 caracteres hexadecimales), la comparación == de CPython se ejecuta en el orden de decenas de nanosegundos, muy por debajo del overhead de medir en Python (llamada a función, creación de objetos), que domina el tiempo total y enmascara la señal — el experimento, tal como está planteado, no tiene resolución suficiente para observar el efecto. Lo que sí queda confirmado de forma clara es que `verificar_firma()` se mantiene constante (~5100-5300 ns) con independencia de la posición del error, consistente con el uso de `secrets.compare_digest()` en lugar de == (ver auditoría estática anterior). Evidencia: `docs/rs4_auditoria_comparaciones.txt`, `docs/rs4_resultados_timing.txt`, `captures/rs4_timing_analysis.png`.

5. Manual de despliegue y ejecución

Paso 1 — Instalar dependencias

```
pip install -r requirements.txt
```

Paso 2 — Sembrar la base de datos (crea secbank.db con los usuarios de prueba alice, bob, charlie)

```
python -m app.seed
```

Paso 3 — Arrancar el servidor (HTTP plano, sin TLS, puerto 8080)

```
uvicorn app.main:app --reload --port 8080
```

Paso 4 — Ejecutar el cliente de demostración (login, transferencia legítima, ataque de integridad, replay)

```
python client_test.py
```

Paso 5 — Tests automatizados (RS2/RS3)

```
pytest test_integrity.py -v
```

Paso 6 — Auditoría y experimento de tiempo constante (RS4)

```
python timing_analysis_rs4.py
```

Para reproducir las capturas de red: abrir Wireshark en la interfaz de loopback, filtro `tcp.port == 8080`, lanzar el servidor (paso 3) y ejecutar `client_test.py` (paso 4) con la captura activa.

6. Matriz de trazabilidad de requisitos

Requisito	Fichero / función	Evidencia
RF1.a/c — Registro, sin duplicados	app/main.py: register()	409 Conflict en username/IBAN duplicado
RF1.b — BD con usuarios de prueba	app/seed.py	alice, bob, charlie precargados
RF1.d — Login y Logout	app/main.py: login(), logout()	ActiveSession creada/borrada
RF2 — Transacción JSON (cuentas origen/destino, amount, concept)	schemas.py, transfers.py	client_test.py, test_integrity.py
RS1.a — Argon2id + salt	security.py: hash_password()	app/security.py (líneas 9-18), pytest test_integrity.py
RS1.b — Rate limiting / bloqueo	main.py: login()	423 Locked tras 5 intentos
RS2.a — Firma HMAC-SHA256	security.py: firmar_mensaje, verificar_firma	02_evidencias_rs2_rs3.pcapng
RS2.b — Clave de 256 bits, CSPRNG	security.py: generate_session_key_256bit	secrets.token_hex(32)
RS3.a/b — Nonce + timestamp + tabla	integrity.py: verificar_transferencia_firmada	02_evidencias_rs2_rs3.pcapng (replay)
RS4.a — Comparación tiempo constante	security.py: verificar_firma	rs4_resultados_timing.txt

7. Uso de herramientas de IA

El equipo ha utilizado Claude (Anthropic) como asistente de desarrollo y documentación, con revisión y validación propia en todos los casos. Tareas concretas:

• Depuración de entorno: resolución de errores de compilación de dependencias en Windows/Python 3.13 y de problemas de escapado de JSON en PowerShell — validado reproduciendo los comandos y comprobando las respuestas HTTP reales del servidor.
• Generación del script de auditoría y experimento empírico de RS4 (`timing_analysis_rs4.py`) — validado ejecutándolo contra las funciones reales del proyecto y revisando que los resultados (tabla y gráfica) fueran coherentes con el comportamiento esperado de `secrets.compare_digest()`.
• Análisis de las capturas de Wireshark (identificación de los paquetes relevantes de login, transferencia legítima, ataque de integridad y replay dentro de una captura con ruido) — validado contrastando manualmente los paquetes señalados con la propia captura en Wireshark.
• Redacción y estructuración de esta memoria a partir del código y las evidencias ya generadas por el equipo — el contenido técnico (código, capturas, resultados) es trabajo propio del equipo; la IA se usó para la redacción y el formato.

El equipo declara ser responsable del contenido íntegro del código y de esta memoria, y poder explicar y defender en detalle cualquier control criptográfico implementado.